INDEPENDENT RESEARCH PROJECT · QUANTUM ERROR CORRECTION

# QEC Decoder Latency Study

Characterising classical feedback latency in software-based quantum error correction — Final Report

**Swayam Shakya**

14 October 2026

## Abstract

A software decoder for the 3-qubit bit-flip code meets a 40 µs coherence deadline in all but about 1 cycle in 100,000; the residual risk is operating-system stalls, not decoder speed.

Real-time quantum error correction needs a classical decoder to return each correction before the qubits decohere. We measured decode latency for the bit-flip code in Python and in C: 10,000 cycles per tier, with timer floors and a ten-run repeat study. We report the violation rate VR(τ) = P(L > τ) with exact binomial intervals. Typical decodes took 113 ns (Python) and 31 ns (C), and VR fell below the 1% budget at 268 ns and 46 ns. At 40 µs, Python missed once in 110,000 cycles and C never did. Both tiers shared a rare plateau of 4–42 µs stalls that no fitted distribution predicts.

On calibrated replicas of IBM Heron r2 and Eagle r3 devices, a dynamic-circuit implementation with real-time feed-forward raised logical fidelity by 1.6–1.7 points over a bare qubit. Inserting a decoder wait doubled the logical error after 8–13 µs, so the 40 µs deadline is a ceiling rather than a safe operating point. All results come from code executed end to end on 14 October 2026; running on physical IBM hardware is the remaining step.

# Contents

**1. Introduction**

**2. Background**
- 2.1 The code
- 2.2 The decoherence deadline
- 2.3 The violation rate

**3. Methods**
- 3.1 Software and machine
- 3.2 Noise models
- 3.3 Experiments
- 3.4 Latency tiers
- 3.5 Statistics

**4. Results — Weekend 1: the physical layer**
- 4.1 Coherence decay (Experiment 1.1)
- 4.2 Pauli sensitivity (Experiment 1.2)

**5. Results — Weekend 2: the code works**
- 5.1 The circuit
- 5.2 Syndrome decoding under noise (Experiment 2.1)
- 5.3 Logical versus physical error (Experiment 2.2)

**6. Results — Weekend 3: latency, jitter and the violation rate**
- 6.1 RQ1: where VR(τ) crosses the 1% budget
- 6.2 RQ2: the shape of the tail
- 6.3 RQ4: Python versus C
- 6.4 A note on interval methods

**7. Results — Weekend 4: the protocol on calibrated device replicas**
- 7.1 The circuits on each device
- 7.2 Fidelity by protocol
- 7.3 Logical error versus decoder latency
- 7.4 Status of the real-hardware run

**8. Discussion**
- 8.1 The deadline is a ceiling, not a target
- 8.2 The average cost is tiny; the tail is the risk
- 8.3 What changing the language buys
- 8.4 Simulation versus device noise (RQ3)
- 8.5 Method lessons

**9. Limitations and threats to validity**

**10. Conclusion and future work**

**Appendix A. Reproducibility**

**Appendix B. Defects found and fixed during verification**

**References**

# 1. Introduction

This project asks whether a classical decoder written in ordinary software can finish every error-correction cycle inside the time a superconducting qubit stays coherent, and how often it misses.

A quantum error-correction (QEC) code only protects information if the loop *measure syndrome → decode → apply correction* closes before the qubits decohere. This project assumes a dephasing time of $T_2$ = 120 µs, a conservative figure for IBM superconducting qubits (the calibrated devices in Section 7 have medians of 144–170 µs). With N = 3 syndrome rounds per logical cycle, each round's classical work must fit in $\tau_{DD} = T_2/N$ = 40 µs. Simulations of QEC usually report logical error rates and treat the classical side as instantaneous; this project measures the classical side directly.

The project ran over four weekends and answers four research questions:

| RQ | Question | Where answered |
|---|---|---|
| RQ1 | What is the violation rate VR(τ) = P(L > τ) across deadlines from 10 ns to 100 µs, and does it cross the 1% surface-code threshold? | Section 6 |
| RQ2 | Is the latency tail light (log-normal, Weibull) or heavy (Pareto-like)? | Section 6 |
| RQ3 | How does realistic device noise change the code's benefit relative to the idealised simulation? | Sections 5 and 7 |
| RQ4 | How much does a compiled C decoder reduce latency and VR relative to pure Python? | Section 6 |

The contributions are a reproducible latency dataset of 10,000 decode cycles per implementation tier, a statistical framework for deadline analysis (exact binomial intervals on VR(τ), bootstrap intervals on $\tau^*$, Anderson–Darling and Kolmogorov–Smirnov fits, the Hill tail index, Mann–Whitney U with effect size), and an end-to-end dynamic-circuit implementation of the 3-qubit bit-flip code that runs through IBM's current client-side Sampler primitive. Every number in this report comes from the code in the project repository, executed for this report; the repository's demo page is `index.html`.

# 2. Background

The 3-qubit bit-flip code stores one logical bit in three physical qubits, checks two parities, and fails only when two or more qubits flip.

## 2.1 The code

The logical states are $|0\rangle_L = |000\rangle$ and $|1\rangle_L = |111\rangle$. Two ancilla qubits measure the parities $Z_0Z_1$ and $Z_1Z_2$ without measuring the data qubits themselves, so a superposition of logical states survives the check. The code corrects any single X (bit-flip) error. It is blind to Z (phase) errors, which is why Experiment 1.2 measures which Pauli errors a state can feel.

Qiskit prints classical bits right to left, so the 2-bit syndrome string reads $(Z_0Z_1)(Z_1Z_2)$. The decoder is a four-entry look-up table:

| Syndrome string | $Z_0Z_1$ | $Z_1Z_2$ | Diagnosis | Correction |
|---|---|---|---|---|
| `00` | 0 | 0 | no error | none |
| `10` | 1 | 0 | qubit 0 flipped | X on $q_0$ |
| `11` | 1 | 1 | qubit 1 flipped | X on $q_1$ |
| `01` | 0 | 1 | qubit 2 flipped | X on $q_2$ |

If each data qubit flips independently with probability p, the logical qubit fails when two or three flip:

$$P_L(p) = 3p^2(1-p) + p^3 = 3p^2 - 2p^3$$

The code therefore beats an unencoded qubit for every p < 1/2, and by a factor of about 1/(3p) when p is small.

### 2.2 The decoherence deadline

A logical cycle of N syndrome rounds must finish within the dephasing time $T_2$. Each round's classical work (decode, choose a correction, dispatch it) therefore has a budget of:

$$\tau_{DD} = \frac{T_2}{N} = \frac{120\,\mu s}{3} = 40\,\mu s$$

### 2.3 The violation rate

For measured decode latencies $L_1 \dots L_n$, the violation rate at deadline τ is the share of cycles that miss it:

$$VR(\tau) = P(L > \tau) \approx \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[L_i > \tau]$$

The project compares VR(τ) with $p_{th} \approx 1\%$, the surface-code threshold of Fowler et al. (2012). This treats a missed deadline as one more error event. That is a modelling assumption: $p_{th}$ is a per-operation physical error rate, so it serves here as an error budget, not as a law that latency must obey.

## 3. Methods

Eight scripts (seven for the experiments, one for the analysis) were run end to end for this report on a current, mutually compatible Qiskit stack. Every random draw is seeded, so the simulated results reproduce exactly. Timing results vary run to run, and a repeat study measures by how much.

### 3.1 Software and machine

| Component | Version or value |
|---|---|
| Python | 3.12.3 (the pinned stack supports 3.11–3.13) |
| qiskit / qiskit-aer / qiskit-ibm-runtime | 2.5.2 / 0.17.2 / 0.50.0 |
| scipy / numpy / statsmodels / matplotlib | 1.17.1 / 2.4.6 / 0.15.0 / 3.11.2 |

| Component | Version or value |
|---|---|
| C compiler | GCC 13.3.0, `-O2 -shared -fPIC` |
| Host | Linux 6.18 x86_64 virtual machine, Intel Xeon at 2.1 GHz, 2 vCPUs |
| Timer | `time.perf_counter_ns` = `clock_gettime(CLOCK_MONOTONIC)`, 1 ns resolution |

## 3.2 Noise models

| Model | Parameters | Used in |
|---|---|---|
| Thermal relaxation (project default) | $T_1$ = 250 µs, $T_2$ = 120 µs, idle gate 50 ns, CNOT 500 ns | 1.1, 2.1 |
| Single Pauli channel on one idle gate | X, Y or Z with probability p = 0–0.5 | 1.2 |
| Code capacity | independent X error with probability p on each data qubit; ideal checks | 2.2 |
| IBM Heron r2 calibration snapshot (`FakeKingston`, 156 qubits) | median $T_1$ = 283 µs, $T_2$ = 144 µs | 4.1, 4.2 |
| IBM Eagle r3 calibration snapshot (`FakeSherbrooke`, 127 qubits) | median $T_1$ = 278 µs, $T_2$ = 170 µs | 4.1, 4.2 |

The calibrated devices have longer $T_2$ than the project default, so the 40 µs deadline is conservative: the device deadlines are 48 µs (Heron) and 57 µs (Eagle).

## 3.3 Experiments

| ID | Script | What it measures | Sample size |
|---|---|---|---|
| 1.1 | `experiment_1_1_coherence.py` | Ramsey decay of $\|+\rangle$; fits $T_2$ | 25 idle times × 4,096 shots |
| 1.2 | `experiment_1_2_pauli_sensitivity.py` | which Pauli errors $\|+\rangle$ can detect | 3 Paulis × 11 values of p × 8,192 shots |
| 2.1 | `bit_flip_code.py` | syndrome → correction accuracy; Python decode time inside the full cycle | 10,000 noisy single-shot cycles |
| 2.2 | `experiment_2_2_logical_error.py` | logical vs physical error rate | 11 values of p × 100,000 shots |
| 3.1 | `fit_distribution.py` | shape and tail of the 2.1 latencies | 10,000 samples |
| 3.2 | `experiment_3_jitter_profiler.py` | decode latency for three tiers, timer floors, VR(τ) | 10,000 timed cycles per tier after 1,000 warm-up; repeat study 10 × 10,000 |
| 4.1 | `experiment_4_real_hardware.py local` | bare vs encoded fidelity; offline vs real-time correction | 3 circuits × 4,000 shots per device |
| 4.2 | `experiment_4_real_hardware.py sweep` | logical error vs decoder wait (0–80 µs) | 9 waits × 2 circuits × 4,000 shots per device |

### 3.4 Latency tiers

- **Python (measured).** A dictionary lookup between two `perf_counter_ns()` calls.
- **C extension (measured).** A 4-entry look-up table compiled with GCC and called through ctypes. It is timed inside C with `clock_gettime(CLOCK_MONOTONIC)`, so Python overhead is excluded.
- **Bare-metal (modelled, not measured).** An ARM Cortex-M7 at 480 MHz with a 41-cycle worst-case execution time (85.4 ns) plus 5% Gaussian jitter. It is shown for scale and excluded from every hypothesis test.
- **Timer floor.** Two back-to-back clock reads with nothing between them, measured for both real tiers. Subtracting it separates the decode cost from the cost of timing it.

### 3.5 Statistics

- **Fidelities and error rates:** Wilson 95% intervals.
- **VR(τ):** exact Clopper–Pearson 95% intervals. When no cycle misses, a bootstrap interval collapses to [0, 0], but the exact upper bound stays honest (0.037% for 0 misses in 10,000).
- **τ*, the deadline where VR(τ*) = 1% (the 99th percentile):** percentile bootstrap with 5,000 resamples, a setting where the bootstrap is reliable.
- **Distribution shape:** Anderson–Darling on log-latency; Kolmogorov–Smirnov distance D against log-normal, gamma and Weibull fits; Hill tail index on the top 5%.
- **Tier comparison:** Mann–Whitney U with rank-biserial $r = 2U_1/(n_1 n_2) - 1$, where $r > 0$ means the first tier is slower.
- **Protocol comparison:** $\chi^2$ test on 2 × 2 success tables.
- **Seeds:** spread with `numpy.random.SeedSequence`, because Aer seeds shot *k* with seed + *k* and nearby integer seeds give overlapping random streams (Appendix B, item 7).

In Experiment 4, all three circuits run on one fixed physical layout, and the bare qubit is data qubit 0's physical qubit. The bare qubit idles for exactly as long as the encoded circuit's encode-and-syndrome stage (3.15 µs on Heron, 6.76 µs on Eagle).

## 4. Results — Weekend 1: the physical layer

The simulation recovers the dephasing time it was given, $T_2 = 121.5 \pm 1.3$ µs against 120 µs, and shows a qubit keeping 86% fidelity at the 40 µs deadline.

### 4.1 Coherence decay (Experiment 1.1)

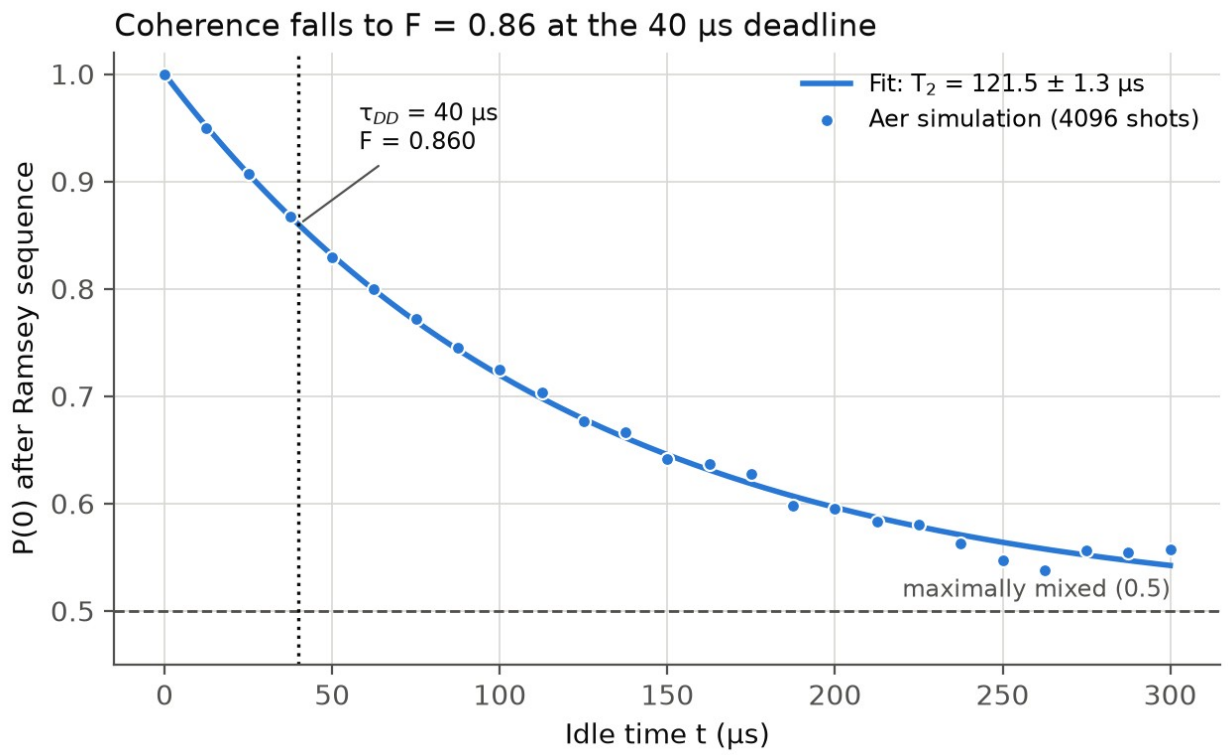



*Figure 1. Ramsey decay: 25 idle times from 0 to 300 µs, 4,096 shots each.*

The one-parameter model $F(t) = 0.5 + 0.5 \cdot \exp(-t/T_2)$ fits all 25 points and returns $T_2 = 121.5 \pm 1.3$ µs. That is 1.2 standard errors from the true 120 µs. At $\tau_{DD} = 40$ µs the fidelity is 0.860. After a full three-round cycle (120 µs) it falls to 0.686, which is why the deadline divides $T_2$ rather than spending it in one round.

The experiment only works because the qubit starts in $|+\rangle$. The original notebook prepared $|0\rangle$, which thermal relaxation never moves. Its fit ran to the optimiser's bound and reported $T_2 = 44{,}142$ µs, 368 times the true value (Appendix B, item 1).

### 4.2 Pauli sensitivity (Experiment 1.2)

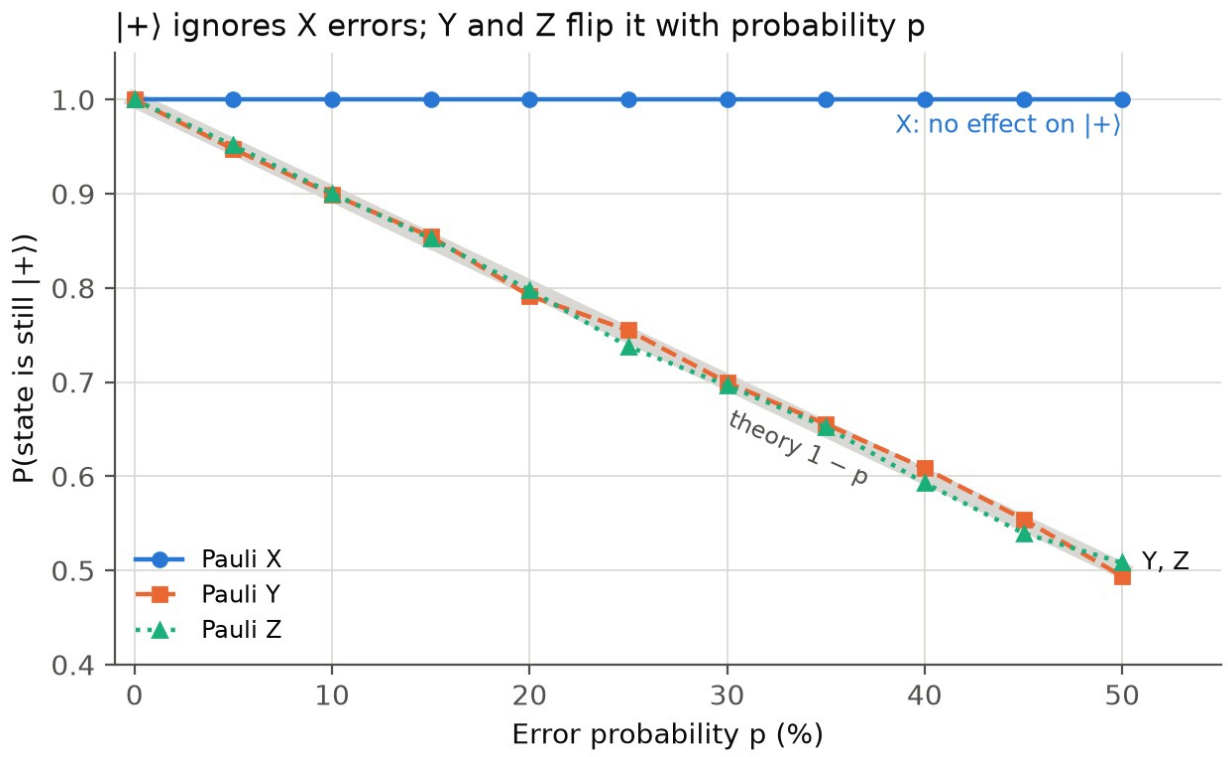



*Figure 2. Survival of |+⟩ under one Pauli channel; 11 values of p, 8,192 shots each.*

The |+⟩ state is immune to X errors: it survived at all 11 error rates with probability 1.000. Y and Z errors flip it with probability p. Every simulated point lies within 0.0123 of the theory line 1 − p; the largest deviation is 2.6 standard errors, at one point out of 22.

This matters for everything after it. The bit-flip code corrects exactly the error that |+⟩ cannot feel, and it cannot see the errors that do harm |+⟩. A code that protects superpositions needs phase-flip checks as well, as in the Shor and surface codes. The original notebook put the noise on both Hadamard gates, so an X error after the second one flipped the readout and made |+⟩ look 49% sensitive to X (Appendix B, item 2).

## 5. Results — Weekend 2: the code works

The code turns a 1% physical bit-flip rate into a 0.039% logical rate, 25 times fewer failures, and its syndrome decoder picks the right correction in 98.61% of noisy cycles.

## 5.1 The circuit

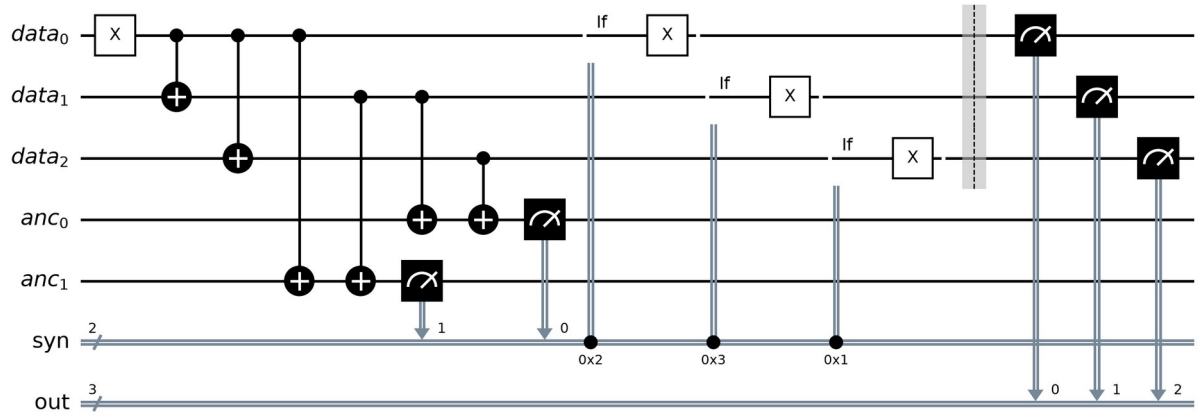



*Figure 3. The 3-qubit bit-flip code with real-time feed-forward, as drawn by Qiskit. The same circuit runs in Experiments 2.2 and 4.*

## 5.2 Syndrome decoding under noise (Experiment 2.1)

Each of 10,000 cycles injected no error or one X error at random, ran one noisy shot, and decoded the syndrome. The decoder chose the matching correction in 9,861 cycles: 98.61% (Wilson 95% CI 98.36–98.82%).

| Injected ↓ / decoded → | none | $q_0$ | $q_1$ | $q_2$ | Correct |
|---|---|---|---|---|---|
| none | **2,499** | 12 | 11 | 12 | 98.62% |
| $q_0$ | 15 | **2,482** | 23 | 7 | 98.22% |
| $q_1$ | 10 | 7 | **2,459** | 10 | 98.91% |
| $q_2$ | 15 | 8 | 9 | **2,421** | 98.70% |

The 139 failures spread evenly over every injected case, so the look-up table has no systematic misassignment. They come from thermal relaxation during the six 500 ns CNOTs: a $|1\rangle$ data qubit decays, or an ancilla is disturbed, and the measured syndrome no longer matches the injected error.

The decode step itself, timed inside the full cycle, took a median of 716 ns (P99 1,193 ns). One cycle in 10,000 took 104 µs and missed the 40 µs deadline; 13 took longer than 10 µs. Section 6 isolates this tail.

### 5.3 Logical versus physical error (Experiment 2.2)

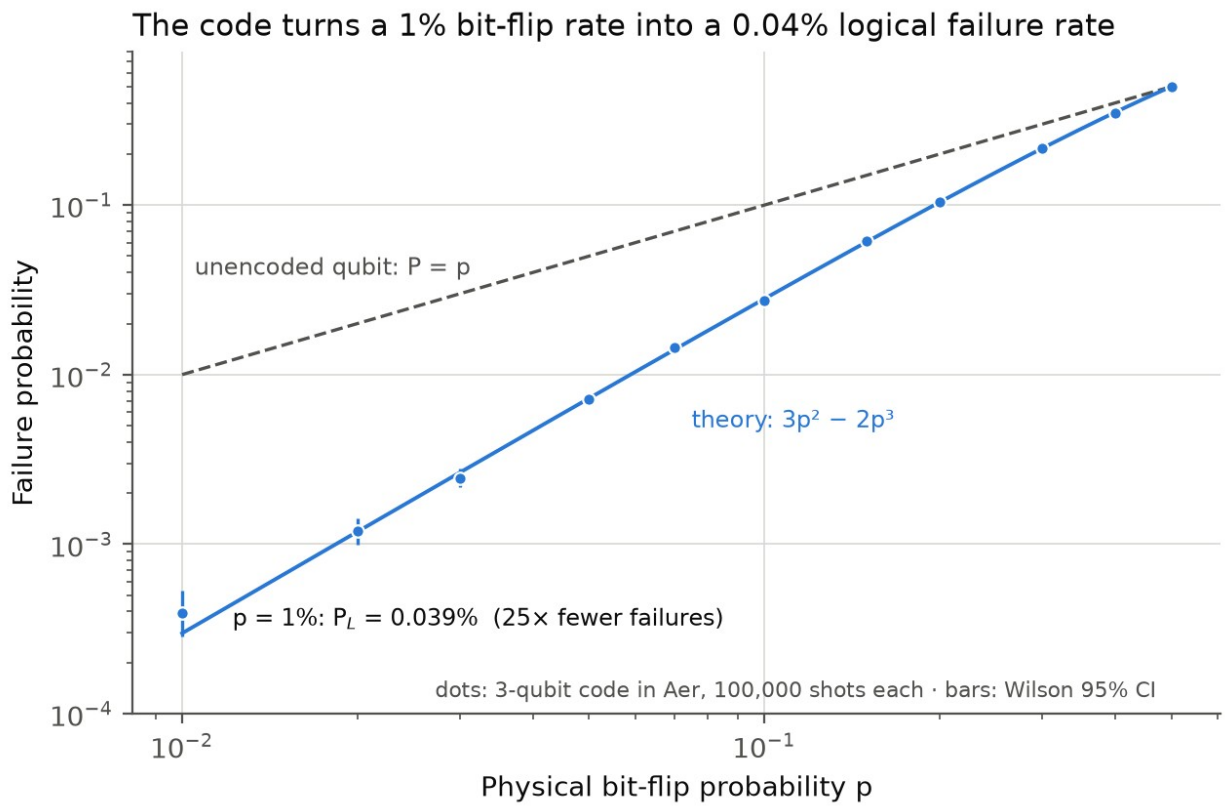



*Figure 4. 11 values of p, 100,000 shots each; bars are Wilson 95% intervals.*

All 11 simulated points contain the theory value $3p^2 - 2p^3$ inside their 95% interval, so the code delivers the quadratic suppression that theory predicts. The benefit shrinks as p grows: 25× fewer failures at p = 1%, 3.6× at 10%, and none at the break-even point p = 50%, where the code and a bare qubit both fail half the time.

## 6. Results — Weekend 3: latency, jitter and the violation rate

A typical decode takes 113 ns in Python and 31 ns in C, 350 and 1,300 times inside the 40 µs budget. But both tiers suffer rare stalls of 4–42 µs, and those stalls, not the decoder, set the violation rate.

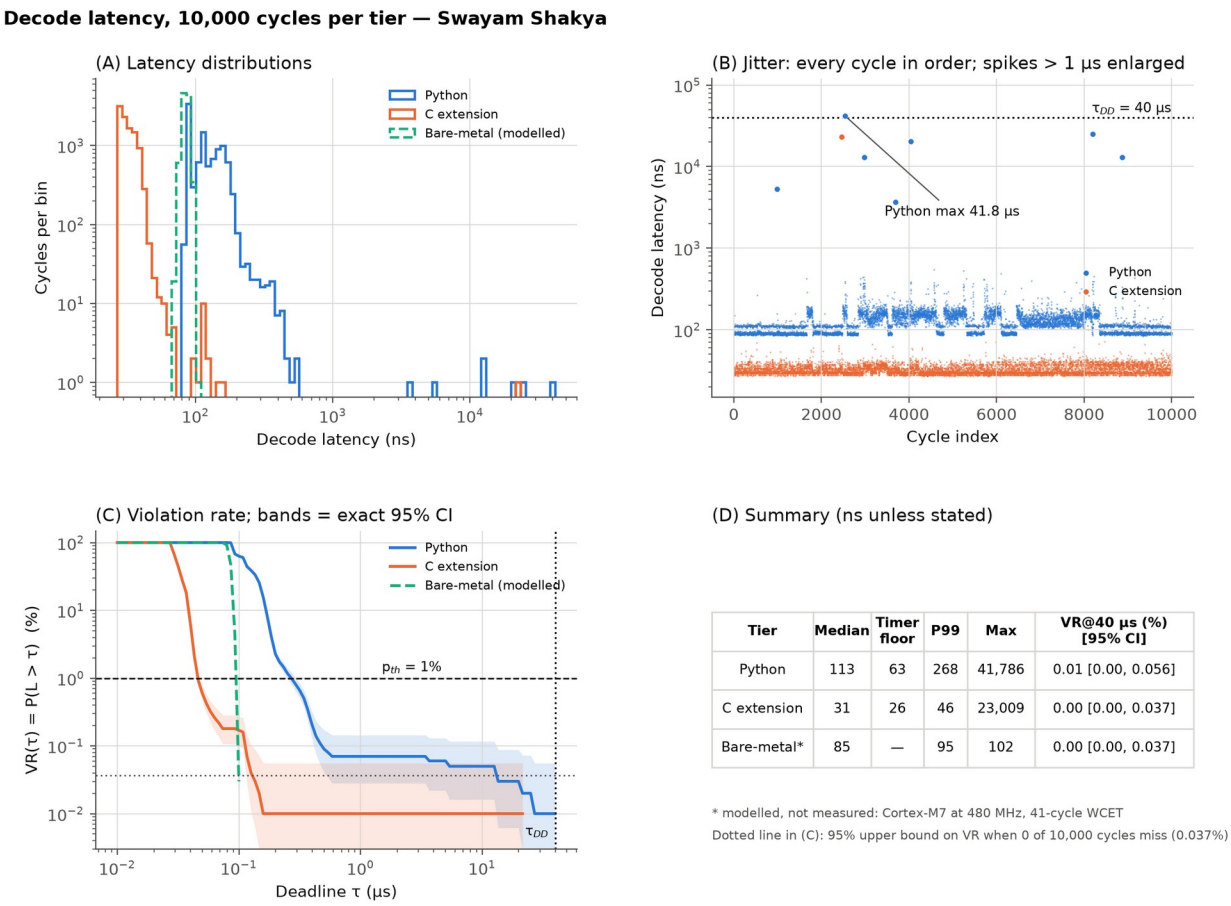



*Figure 5. Decode latency, 10,000 timed cycles per tier after 1,000 warm-up cycles. Bare-metal is modelled, not measured.*

| Tier | Median (ns) | Timer floor (ns) | τ* = P99 (ns), 95% CI | Max (µs) | Misses at 40 µs | VR@40 µs, 95% CI | Hill α |
|---|---|---|---|---|---|---|---|
| Python | 113 | 63 | 268 [248, 290] | 41.8 | 1 of 10,000 | 0.01% [0.0003, 0.056] | 3.95 |
| C extension | 31 | 26 | 46 [45, 47] | 23.0 | 0 of 10,000 | 0% [0, 0.037] | 7.93 |
| Bare-metal (modelled) | 85 | — | 95 [95, 96] | 0.10 | 0 of 10,000 | 0% [0, 0.037] | 52.7 |

## 6.1 RQ1: where VR(τ) crosses the 1% budget

VR(τ) falls below 1% at $\tau^* = 268$ ns for Python and 46 ns for C. That leaves 149× and 870× headroom against $\tau_{DD}$. At the deadline itself, Python missed once in 10,000 cycles (41.8 µs) and C never did.

A repeat study added 10 further runs of 10,000 cycles per tier. Neither tier missed 40 µs in those 100,000 cycles. Pooled over all 110,000 cycles, Python's VR@40 µs is 0.0009%, with a 95% upper bound of 0.0051%, roughly 200 times below $p_{th}$. The runs differed in their typical speed (Python medians 113–148 ns) and in their worst stall (Python 11.4–41.8 µs, C 4.2–26.5 µs). On a shared virtual machine, single-run maxima are not reproducible numbers.

## 6.2 RQ2: the shape of the tail

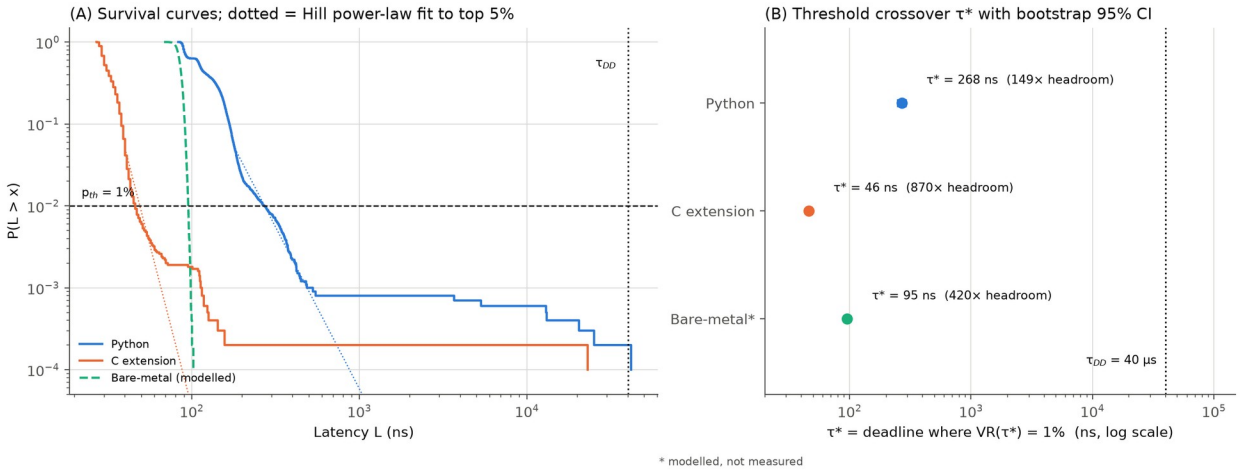



*Figure 6. Tail diagnostics for the Experiment 3.2 run.*

The latency distribution has two parts. The body and near tail are skewed and closest to log-normal by KS distance (D = 0.103 for the Experiment 2.1 data, against 0.177 for gamma and 0.333 for Weibull). Anderson–Darling still rejects exact log-normality ($A^2 = 264$ for Python, $p \le 0.01$). The Hill index on the top 5% is 3.95 for Python, a moderate tail, and 7.93 for C, a light one.

Below a survival probability of about $10^{-3}$, the curves leave the Hill fit and flatten into a plateau of stalls. 7 Python cycles and 1 C cycle took longer than 1 µs, with medians of 13.1 µs and 23.0 µs: 27 and 204 times each tier's own P99.9. Because C shows the same stalls without an interpreter, and the timed Python loop creates no objects the garbage collector tracks, the likely source is the operating system: scheduler preemption, interrupts and hypervisor steal time on a 2-vCPU VM. Neither the log-normal body nor the power-law tail predicts these events, so VR(τ) at large τ must be measured directly, not extrapolated from a fit.

## 6.3 RQ4: Python versus C

C is faster on almost every cycle. Mann–Whitney U gives $p < 10^{-300}$ with rank-biserial r = +0.998, and the median ratio is 3.6×. Subtracting each tier's timer floor leaves a net decode cost of about 50 ns in Python and 5 ns in C. Most of what both tiers measure is the clock.

C does not remove the stalls, though. Its worst cycle (23.0 µs) is the same order as Python's (41.8 µs). Moving the decoder to C improves the typical case 3.6-fold and leaves the deadline risk almost unchanged. Removing that risk needs an environment without preemption: a real-time OS, an isolated core, or the FPGA/ASIC decoders that sit beside quantum control hardware. The modelled Cortex-M7 is slower than C on a Xeon (85 vs 31 ns) but bounded: its worst case is 102 ns.

## 6.4 A note on interval methods

With one observed miss at 40 µs, the percentile bootstrap gives VR in [0, 0.03%], while the exact Clopper–Pearson interval is [0.0003, 0.056%]. The bootstrap's upper bound is 46% too low because resampling cannot invent misses it never saw. With zero misses it collapses to [0, 0] entirely. For rare-event rates, use the exact interval.

# 7. Results — Weekend 4: the protocol on calibrated device replicas

On replicas of IBM Heron r2 and Eagle r3 devices, the code raises logical $|1\rangle$ fidelity from 97.9% to 99.6% and from 97.4% to 99.0%. Real-time and offline correction tie, and inserting a decoder wait doubles the logical error after 8–13 µs.

No IBM account was used. The `local` mode runs the exact transpiled circuits that `submit` would send, under Aer noise models built from the two devices' published calibration snapshots (`FakeKingston`, `FakeSherbrooke`). Running on the physical processors is the one remaining step (Section 7.4).

## 7.1 The circuits on each device

| Device replica | Median $T_1$ / $T_2$ (µs) | Device $\tau_{DD}$ (µs) | Encode + syndrome stage (µs) | Physical qubits | Native 2-qubit gates | Depth (offline / real-time) |
|---|---|---|---|---|---|---|
| Heron r2 (`FakeKingston`) | 283 / 144 | 48 | 3.15 | 108, 109, 110, 118, 129 | 12 CZ | 22 / 26 |
| Eagle r3 (`FakeSherbrooke`) | 278 / 170 | 57 | 6.76 | 41, 53, 59, 60, 61 | 12 ECR | 43 / 46 |

The logical circuit has 6 CNOTs, but the transpiled one has 12 two-qubit gates. The parity checks form a triangle of interactions, and the heavy-hex lattice has none, so the router adds two SWAPs.

## 7.2 Fidelity by protocol

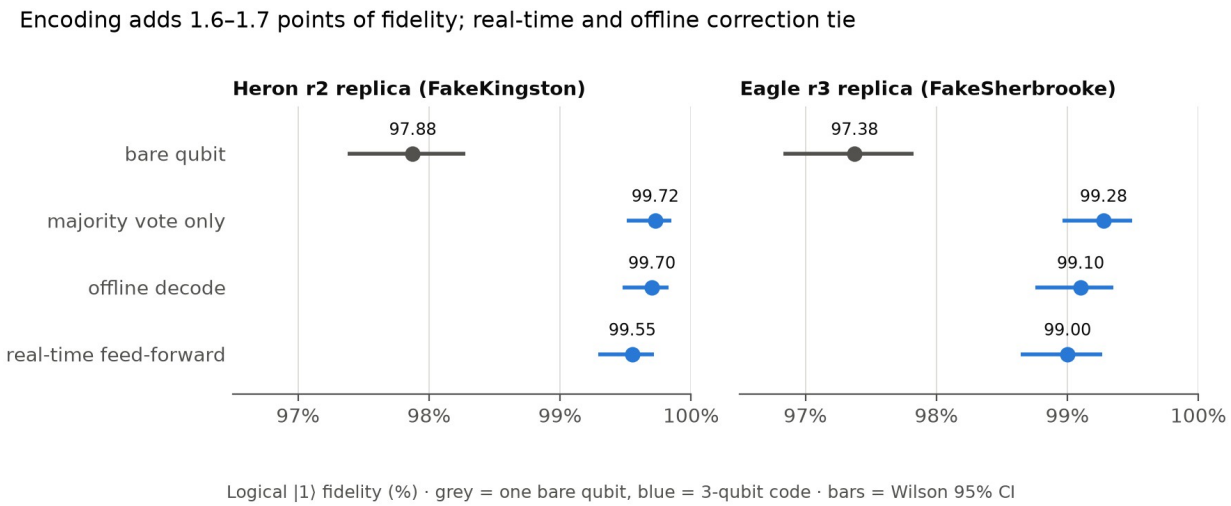



*Figure 7. Logical $|1\rangle$ fidelity by protocol on the Heron r2 and Eagle r3 replicas; 4,000 shots per circuit, bars are Wilson 95% intervals.*

Encoding helps clearly on both devices. Real-time correction beats the bare qubit by 1.68 points on Heron ($\chi^2 = 42.8$, $p = 6 \times 10^{-11}$) and by 1.62 points on Eagle ($\chi^2 = 28.8$, $p = 8 \times 10^{-8}$).

Real-time and offline correction are statistically tied: −0.15 points ($p = 0.36$) on Heron and −0.10 points ($p = 0.73$) on Eagle. This is expected from the replica, which attaches noise to instructions. A qubit idling while the

controller decides on a correction gains no error unless that wait is written into the circuit, which is what Section 7.3 does.

Majority voting alone matches or beats syndrome decoding (99.72% vs 99.70% on Heron, 99.28% vs 99.10% on Eagle). In a single round that ends with reading out the data qubits, a majority vote already undoes any single flip. The syndrome only adds value when the logical state must survive further rounds, and here an ancilla readout error can only cause a wrong correction.

An early version let the transpiler place the offline and real-time circuits on different qubits. That run showed a −0.42-point gap on Eagle, which vanished once both circuits shared one layout (Appendix B, item 10).

## 7.3 Logical error versus decoder latency




*Figure 8. Logical error versus decoder wait on both replicas; 4,000 shots per point. Diamonds mark where the error doubles from its zero-wait value.*

A wait inserted between the syndrome readout and the correction makes the feedback window visible to the noise model. Logical error starts at 0.45% (Heron) and 1.00% (Eagle) and doubles after a 7.6 µs and 12.8 µs wait. By 40 µs it reaches 5.6% and 5.7%, 12.5× and 5.7× its zero-wait value. The code still beats a bare qubit idling for the same time at every wait, by 2.8× on Heron at 40 µs (5.6% vs 15.6%).

Read against Section 6, the typical decoder (31–113 ns) costs nothing measurable. On the Heron replica, a cycle hit by one of the observed stalls, 23 µs in C or 42 µs in Python, would see its logical error rise to roughly 2.5–6%, five to thirteen times the baseline; on Eagle the rise is three- to six-fold.

## 7.4 Status of the real-hardware run

The `submit` and `retrieve` modes are implemented against the current IBM Quantum Platform API, but they need the author's credentials, so they were not run for this report. They pick the least-busy device that supports dynamic circuits and send one job of three circuits × 4,000 shots. The free Open Plan includes 10

minutes of quantum-computer time per 28 days (IBM migration guide), which comfortably covers a job this size. On hardware, the real-time-vs-offline comparison will include the controller's true feed-forward latency, which the replica cannot.

# 8. Discussion

The decoder is not the bottleneck; the computer it runs on is. A look-up-table decode costs 5–50 ns of real work, but a general-purpose host occasionally stalls for up to 42 µs, and on the Heron replica a stall of 10–42 µs multiplies the affected cycle's logical error three- to thirteen-fold.

## 8.1 The deadline is a ceiling, not a target

By the project's own criterion, software decoding passes comfortably. The pooled VR@40 µs is at most 0.0051% (95% upper bound), about 200 times below the 1% budget. But the latency sweep shows the 40 µs deadline is far too lenient as a safety margin. On the Heron replica, logical error is 1.4× its zero-wait value after a 5 µs wait, 2.8× after 10 µs and 12.5× at 40 µs. A cycle that finishes "on time" at 30 µs has already lost most of the code's benefit.

A more useful budget is the wait at which logical error grows by a tolerable factor. Keeping the increase under about 40% requires the whole feedback loop to finish within about 5 µs on this device. That loop includes readout, data transfer, decoding and dispatch, not just the decode measured here.

## 8.2 The average cost is tiny; the tail is the risk

Combining the two halves of the study gives an estimate of the average damage. Weighting the sweep curve by this run's seven Python stalls (3.7–41.8 µs) adds about 0.0012 percentage points of logical error per cycle on average, 0.26% of the 0.45% baseline. Averaged over cycles, stalls are negligible.

The danger is concentration, not the average. A computation of a million cycles would meet several hundred stalls, each one degrading a single cycle by up to 13-fold, and nothing in a general-purpose operating system bounds how long the worst one lasts. Real-time control needs a guaranteed worst case, which is why production decoders run on FPGAs or dedicated cores beside the control electronics.

## 8.3 What changing the language buys

The implementation tier matters less than the execution environment (RQ4). C cut the typical decode 3.6-fold (10-fold net of the timer floor) and made no measurable difference to VR at 40 µs, because both tiers share the same operating-system stalls. For deadline compliance, the next step after C is not a faster language but a quieter machine: an isolated core, a real-time kernel, or hardware.

## 8.4 Simulation versus device noise (RQ3)

Realistic calibrated noise preserves the code's benefit: +1.68 and +1.62 points on the two replicas, both significant at $p < 10^{-7}$. What the replicas cannot show is the controller's own feed-forward delay, the very quantity this project is about. The sweep in Section 7.3 is a proxy for it, and the hardware run is the measurement that would replace the proxy.

### 8.5 Method lessons

Four choices changed results enough to matter for anyone repeating this work. Exact binomial intervals were needed for rare events, where the bootstrap under-covers. Seeds had to be spread, because nearby Aer seeds share random streams. Layouts had to be pinned, because qubit placement alone moved one comparison by 0.4 points. And the timer floor had to be subtracted, because most of what both tiers measured was the clock.

## 9. Limitations and threats to validity

The results are strongest for the simulations, which reproduce exactly, and weakest for anything that depends on one machine's timing or on a device the study did not touch.

- **No physical QPU run.** Experiment 4 used calibration snapshots, not live processors. The snapshots are fixed in time while real devices drift, and Eagle is no longer among IBM's current processor families.
- **The replicas omit timing noise.** Aer attaches noise to instructions, so idling during another qubit's measurement and the controller's feed-forward delay add no error unless written in as delays. The sweep's inserted wait is a proxy for latency, not a measurement of it.
- **Decode only.** The latencies cover the syndrome-to-correction lookup. Readout, transfer to the classical processor and dispatch of the correction, which dominate a real feedback loop, are outside the measurement.
- **One host.** All timing comes from a 2-vCPU cloud virtual machine. Stall frequency and length depend on the host, its load and its kernel, so a laptop or a dedicated server will differ. The repeat study shows that single-run maxima vary from 4 to 42 µs.
- **Bare-metal is modelled.** The Cortex-M7 tier is a Gaussian around a hand-computed WCET. It sets a scale and supports no statistical conclusion.
- **A minimal code.** The bit-flip code corrects only X errors, runs a single round, and here protects a basis state. Surface-code decoders (matching, union-find) cost far more per round and scale with code distance, so the latency margins found here do not transfer to them.
- **The threshold analogy.** Comparing VR(τ) with $p_{th} \approx 1\%$ treats a missed deadline as one more error event. It is a convenient budget, not a derived bound.
- **Fitted tests.** KS p-values after fitting are optimistic, and the Hill index depends on the 5% tail fraction chosen. The conclusions rest on the survival curves and on direct counts of misses, not on either statistic alone.

## 10. Conclusion and future work

Software decoding of the 3-qubit bit-flip code meets a 40 µs deadline in all but about 1 cycle in 100,000. The remaining risk comes from operating-system stalls rather than decoder speed, and logical error starts climbing at waits far shorter than the deadline.

| Question | Answer |
|---|---|
| RQ1: Does VR(τ) cross 1% inside the deadline range? | Yes, at $\tau^* = 268$ ns (Python) and 46 ns (C). At 40 µs, Python missed 1 cycle in 110,000 and C missed none: VR ≤ 0.0051% and ≤ 0.0034% (95% upper bounds). |
| RQ2: Is the tail light or heavy? | Two-part. The body and near tail are skewed and roughly log-normal, with Hill α = 3.95 (Python) and 7.93 (C). Beyond P99.9, a separate plateau of 4–42 µs operating-system stalls appears that no fitted distribution predicts. |
| RQ3: Does device noise change the code's benefit? | The benefit survives: +1.7 and +1.6 points of fidelity on Heron r2 and Eagle r3 replicas ($p < 10^{-7}$). Each microsecond of feedback wait erodes it, and the error doubles after 8–13 µs. |
| RQ4: How much does C help? | 3.6× on the median (10× net of the timer floor), with no measurable change in VR at 40 µs, because both tiers share the same stalls. |

Five steps would carry the work forward:

1. Run `experiment_4_real_hardware.py submit` on a Heron device to measure the true cost of real-time feed-forward against offline decoding.
2. Extend to several syndrome rounds on a superposition state, where real-time correction matters and majority voting no longer suffices.
3. Time the whole feedback loop on the controller, not just the decode, using the device's own timestamps.
4. Repeat the latency study on an isolated core with a real-time kernel to test whether the stall plateau disappears.
5. Replace the look-up table with a surface-code matching decoder and measure how its latency tail grows with code distance.

## Appendix A. Reproducibility

Every script lives in the project repository under `src/`, and the demo page `index.html` shows each one with a copy button; the page's code blocks are generated from the same files. Clone the repository and run two commands:

```
bash setup.sh     # creates qec-env and installs the pinned stack
bash run_all.sh   # runs every experiment into outputs/, about 6 minutes on 2 vCPUs
```

The reference run's files are kept in `reference_results/`, and `pytest` runs 32 unit tests of the decoder, the C library, the statistics and the Weekend 4 circuits. Simulated results reproduce exactly, because every Aer seed is fixed and spread with `SeedSequence`. A clean re-run from the notebook on Python 3.13 matched this report's confusion matrix and logical-error values digit for digit. Timing results reproduce only approximately: the same re-run measured a median in-cycle decode of 881 ns against 716 ns here.

| Output file | Produced by | Contents |
|---|---|---|
| `w1_coherence.json`, `fig1_coherence_decay.pdf` | Experiment 1.1 | Ramsey data, fitted $T_2$ |
| `w1_pauli.json`, `fig2_pauli_sensitivity.pdf` | Experiment 1.2 | Survival of $\|+\rangle$ per Pauli and p |
| `w2_latencies.json` | Experiment 2.1 | 10,000 decode latencies, confusion matrix |
| `w2_logical_error.json`, `fig2b_logical_error.pdf` | Experiment 2.2 | Logical vs physical error with Wilson intervals |
| `w3_fit.json` | Experiment 3.1 | AD, KS and Hill statistics |
| `w3_latencies.json`, `fig3_main_result.pdf` | Experiment 3.2 | Raw latencies and timer floors per tier, VR curves, host details |
| `w3_repeats.json` | Experiment 3.2 `--repeats 10` | Per-run medians, tails and misses; pooled VR |
| `paper_stats.json`, `table1.tex`, `fig4_tail_diagnostics.pdf` | Analysis | $\tau^*$ with bootstrap intervals, Mann–Whitney, LaTeX Table 1 |
| `w4_results_<device>.json`, `fig5_hardware_<device>.pdf` | Experiment 4.1 | Fidelities, $\chi^2$ tests, transpiled-circuit details |
| `w4_sweep_<device>.json`, `fig6_latency_sweep_<device>.pdf` | Experiment 4.2 | Logical error vs decoder wait |

## Appendix B. Defects found and fixed during verification

Each script from the previous notebook was extracted, executed and checked against theory. These defects surfaced, and all are fixed in the delivered notebook.

| # | Where | Defect | Fix |
|---|---|---|---|
| 1 | Exp 1.1 | Prepared $\|0\rangle$, which relaxation never moves; fit reported $T_2$ = 44,142 µs (368× the input) | Ramsey on $\|+\rangle$, noise on idle gates only |
| 2 | Exp 1.2 | Noise on both Hadamards made $\|+\rangle$ look 49% sensitive to X | One noisy idle gate |
| 3 | fit_distribution.py | `kstest(lat, 'weibull')`: no such SciPy distribution, crash | Pass each distribution's `cdf` |
| 4 | fit_distribution.py | Contradictory tail thresholds (α > 2 vs α > 4) | One three-band rule |
| 5 | Weekend 3, Analysis | `anderson()` critical values deprecated in SciPy 1.17, removed in 1.19 | `method='interpolate'` p-values |
| 6 | analysis_paper_ready.py | Five `ff"…"` syntax errors; all warnings silenced; promised figure missing; effect-size sign undocumented | Rewritten |
| 7 | All Aer runs | Nearby seeds share random streams (correlated 3σ deviations); fixed class seed replayed identical noise | `SeedSequence`; fresh seed per cycle |
| 8 | Profiler | Substituted invented C data if the library was missing; duplicated bootstraps; VR = 0 on log axes; threshold line in a series colour | Error on missing library; fixed plotting |
| 9 | VR intervals | Bootstrap collapses to [0, 0] with no misses | Exact Clopper–Pearson |
| 10 | Exp 4 | `id` padding deleted by the transpiler; offline and real-time circuits on different qubits | Duration-matched delay; one pinned layout |
| 11 | Exp 4 | Removed `ibm_quantum` channel; placeholder token in `retrieve`; deprecated `SamplerV2` | `ibm_quantum_platform`, saved account, client-side `Sampler` |
| 12 | requirements.txt | Qiskit 1.3 / runtime 0.23 cannot reach IBM's current platform; matplotlib 3.9.1 yanked | Current pins, `pip check` clean |
| 13 | qec_decode.c | Failed under `-std=c11` (`clock_gettime` undeclared) | `_POSIX_C_SOURCE 200809L` |
| 14 | Notebook page | Bare < in code, no charset or doctype, wrong theme toggle on light systems, unreadable light-mode code colours, placeholder abstract | Generated from tested files; fixed |

## References


1. Fowler, A. G., Mariantoni, M., Martinis, J. M. & Cleland, A. N. (2012). Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A* 86, 032324. doi:10.1103/PhysRevA.86.032324
2. Terhal, B. M. (2015). Quantum error correction for quantum memories. *Rev. Mod. Phys.* 87, 307.
3. Nielsen, M. A. & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*, 10th anniversary edition. Cambridge University Press.
4. Hill, B. M. (1975). A simple general approach to inference about the tail of a distribution. *Ann. Statist.* 3(5), 1163–1174.
5. Clopper, C. J. & Pearson, E. S. (1934). The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika* 26(4), 404–413.
6. Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *J. Am. Stat. Assoc.* 22(158), 209–212.
7. Mann, H. B. & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Ann. Math. Statist.* 18(1), 50–60.
8. Javadi-Abhari, A. et al. (2024). Quantum computing with Qiskit. arXiv:2405.08810.
9. IBM Quantum. Migrate from Classic to the new IBM Quantum Platform. Channel `ibm_quantum_platform`, sunset of the classic platform on 1 July 2025, Open Plan allowance. Accessed 14 Oct 2026.
10. IBM Quantum. qiskit-ibm-runtime release notes. Removal of the `ibm_quantum` channel (0.41.0), deprecation of `SamplerV2` (0.50.0). Accessed 14 Oct 2026.
11. IBM Quantum. Processor types. Current Heron and Nighthawk families. Accessed 14 Oct 2026.